



操作系统课程实验

——从0到1的小型内核实现

2025年 秋季学期

lab-0 实验简介与环境配置

1. 我们要做什么
2. 为什么做这个
3. 你需要知道的事
4. 实验环境配置
5. 你应该学习的前置知识

1. 我们要做什么

- 我们要做一件很酷的事情——从0到1开发一个小型操作系统内核。
- **没概念？** 就是写一个程序，不过它应该比你接触过的任何程序都更加复杂和强大。
- **什么叫从0到1？** 你不需要改动任何人写的代码，你面对的是一张白纸。
- **我知道，我们写的C语言算法题就是这样！** 不，在这里你几乎不能调用任何库。
- **不调用库函数怎么编程？连打印字符串都做不到！** 不必担心，做完lab-1就能了.....
- **我们真的要平地起高楼吗？毫无头绪啊！** 两个好消息：①助教会通过实验文档给你建立逻辑框架（虽然不一定足够完善清晰）②你可以看到隔壁高楼建完的样子—xv6
- 从易于理解的角度出发，我们研究的是：如何从0开始建立一个类似xv6的OS内核

注：你可以参考的xv6版本是xv6-riscv 2020版，包括代码、文档、其他人的解析

2. 为什么做这个

回答很简单：因为你是没有经验的初学者，做一个简单的demo是很好的学习路径。

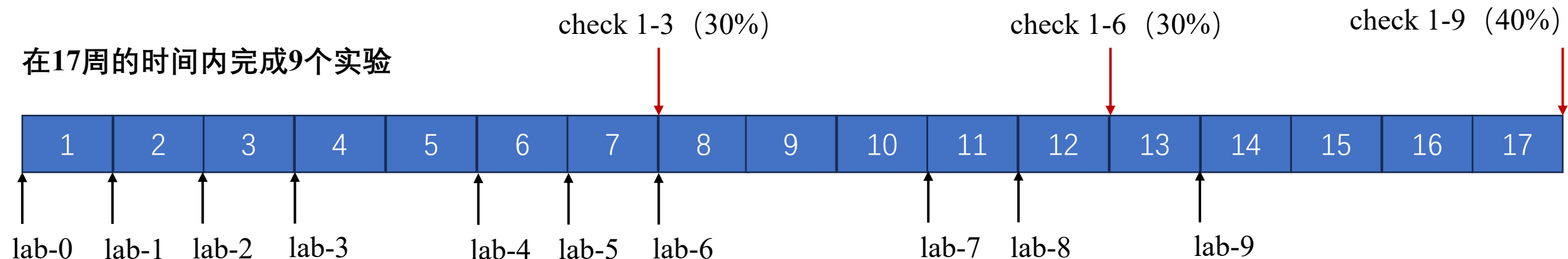
学操作系统，就去写一个小型操作系统；学体系结构，就去搭一个小型CPU；学编译原理，就去写一个小型编译器.....在计算机系统领域，这是一种通用策略。

具体来说，你会有如下收获：

1. 对操作系统的运行机制和常见策略形成深刻理解
2. 清楚一些常见操作背后的魔法，例如malloc、printf、exit等
3. 从单文件算法编程走向多文件系统编程
4. 理解软件开发的流程和思想，显著提升工程能力
5. 初步了解git、makefile、qemu、gdb等实用工具
6. 更强的心理韧性，面对失败百折不挠

3. 你需要知道的事

在17周的时间内完成9个实验



助教会在3个Checkpoint检查你的代码仓库，具体包括：

1. git 提交记录 → 规律稳定的提交、不存在抄袭等异常行为
2. 文档 → 本次实验的完成度、对本次实验的理解、清晰可读
3. 代码 → 符合规范的代码、有必要的注释、能通过测试

Checkpoint-1: 依据lab 1-3的完成情况进行打分（占总分30%）

Checkpoint-2: 依据lab 1-6的完成情况进行打分（占总分30%）

Checkpoint-3: 依据lab 1-9的完成情况进行打分（占总分40%）

为什么这么设计？

- 不是所有人都能在17周内完成9个实验
- 考虑做的快的学生，我们尽早发布实验
- 考虑做的慢的学生，我们延迟检查实验
- 尽量保障时间弹性，我们只检查三次
- 如果遇到放假，实验会正常发布，但是实验课的讲解延后一周
- 如果大家普遍做不完，检查点可以延后

3. 你需要知道的事

考虑到实验的工作量和难度，我们鼓励学生组成两人小队：

助教会建立一个共享文档，请大家在第1周完成组队，从第2周开始就不可变更
填写“队员-1、队员-2、代码仓库链接、实验完成度（0~9）”

同队2人原则上得分相同，如有特殊情况可能对其中1人酌情扣分

如果你不幸落单或主动选择单人完成，原则上不会获得加分优势（可能有印象分？）

关于答疑：

1. 助教会建立一个实验交流群，供学生互相讨论实验中遇到的问题 + 反映实验的代码、测试、文档中不清楚/不完善/错误的地方
2. 线下答疑时间：周一实验课 + 周一晚上18:00 ~ 21:00（理科大楼 B230最里面的一排）
3. 由于助教事务缠身，请不要加助教的私人微信，遇到问题请通过前两条渠道反映

注：答疑时间可能会修改，如有修改会及时通知大家

3. 你需要知道的事

实验的基本原则：

- 1. 开源但是不要抄袭：**所有小队的线上仓库设为公开，地址填入共享文档，大家可以很方便地查看其他组的代码、文档、进展。我们希望以此帮助和激励能力薄弱的小组；但是请不要抄袭，你的每次提交都有记录，如果助教第一次发现异常会进行警告并扣分，第二次发现异常实验最终得分不会超过60。
- 2. 参考但是不要复制：**xv6的原版实现是所有小队的共同参考依据，但请不要复制它的代码。一方面，直接复制无法完成实验要求；另一方面，你会错过很好的学习机会。因此，我们建议你阅读代码、理解逻辑、自己动手实现。
- 3. 尽力而为：**坦诚地说，我们的实验是具有挑战性和选拔性的，我们希望找到真正热爱操作系统的人。对于能力平凡/不感兴趣的同学，尽力而为完成实验即可，我们会综合考虑多方因素并给出合理的分数（较高的基础分 + 可控的分差）。
- 4. 作品而非作业：**全部9个实验完成后，你会实现一个作品作为这门课的纪念 / 深造道路的敲门砖。因此，好好对待你的代码，认真完成你的内核，而不是写完作业就完了。

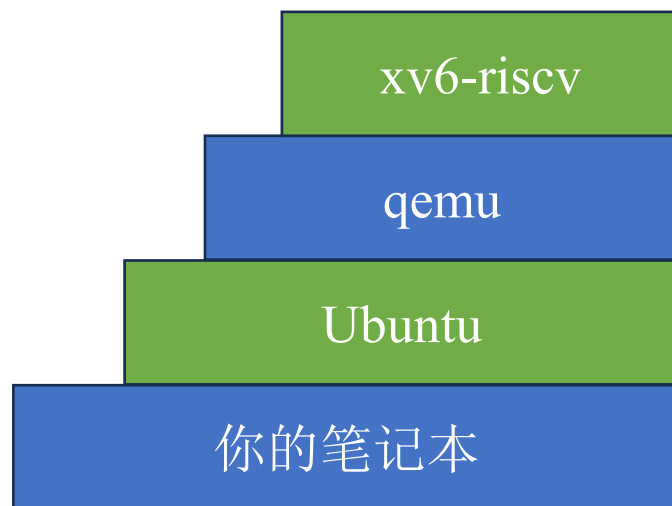
4. 实验环境配置

 操作系统（软件）

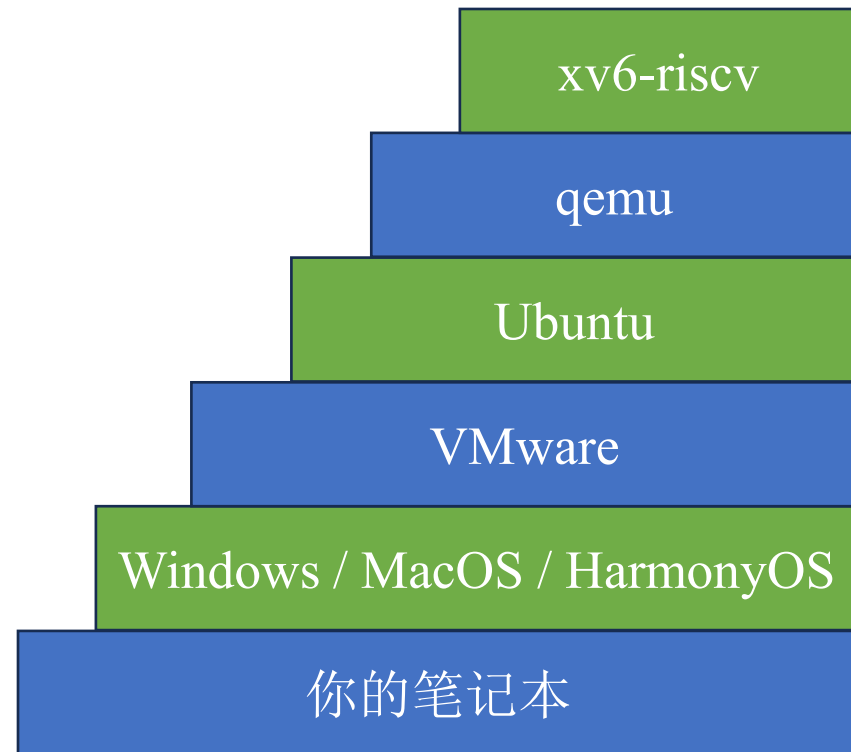
 硬件/模拟硬件



符合直觉。



不太常见？



太抽象了！

我们的教程基于
第三种运行环境
核心软件版本：

1. VMware Workstation pro 17
2. Ubuntu 22.04
3. qemu 5.1.0
4. xv6-riscv-2020

实验环境配置成功的标志：

xv6-riscv-2020能够成功运行

注：更多细节请参考配套的Word文档

5. 你应该学习的前置知识

1. **Git 与 Gitee:** 学习git的简单用法，在gitee上建立一个公开的空白的代码仓库，它需要包含一个README文件用于介绍你的小组成员和你的项目，当然也可以包括.gitignore和许可证（线上仓库最晚在第二周内建好）
2. **README:** 学习简单的Markdown语法，以后你写文档会用得着。
3. **Makefile与程序编译:** 能够在AI的帮助下熟悉Makefile的工作原理，以及背后的程序编译、链接、运行的基本原理
4. **基于C语言的包括多个源文件的项目:** 理解多个.c、.h、.S等各种类型源文件是如何协作的
5. **Linux命令行:** 在AI的帮助下熟悉Linux环境下的常用命令，理解Linux系统的基本概念
6. **xv6-riscv:** 对xv6的原理建立初步认识，学习编译和运行xv6，探索它的玩法

未来的17周注定不轻松，早点开始，玩得开心！